Deadline-Constrained Cost Optimization Approaches for Workflow Scheduling in Clouds

Quanwang Wu, Fuyuki Ishikawa, *Member, IEEE*, Qingsheng Zhu, Yunni Xia, *Senior Member, IEEE*, and Junhao Wen

Abstract—Nowadays it is becoming more and more attractive to execute workflow applications in the cloud because it enables workflow applications to use computing resources on demand. Meanwhile, it also challenges traditional workflow scheduling algorithms that only concentrate on optimizing the execution time. This paper investigates how to minimize execution cost of a workflow in clouds under a deadline constraint and proposes a metaheuristic algorithm L-ACO as well as a simple heuristic ProLiS. ProLiS distributes the deadline to each task, proportionally to a novel definition of probabilistic upward rank, and follows a two-step list scheduling methodology: rank tasks and sequentially allocates each task a service which meets the sub-deadline and minimizes the cost. L-ACO employs ant colony optimization to carry out deadline-constrained cost optimization: the ant constructs an ordered task list according to the pheromone trail and probabilistic upward rank, and uses the same deadline distribution and service selection methods as ProLiS to build solutions. Moreover, the deadline is relaxed to guide the search of L-ACO towards constrained optimization. Experimental results show that compared with traditional algorithms, the performance of ProLiS is very competitive and L-ACO performs the best in terms of execution costs and success ratios of meeting deadlines.

Index Terms—Ant colony optimization, cloud computing, constrained optimization, deadline, workflow scheduling

1 Introduction

Nowadays, scientific and business applications can contain hundreds or thousands of computing tasks. The workflow model is extensively applied to represent these applications via direct acyclic graphs (DAGs), where nodes represent tasks and edges represent dependencies among them [1]. These large-scale workflow applications are deployed in a distributed computing environment in order to execute the workflows in a reasonable amount of time. However, traditional distributed computing platforms, such as high-performance clusters and grid systems, are not only very expensive to build and maintain, but also inefficient in adapting to the surge of resource demands. With the emergence of cloud computing and the rapid deployment of cloud infrastructure, more and more large-scale workflow applications have been moved or are in active transition to the cloud, where users can dynamically acquire and release cloud resources on demand, and pay based on the usage.

Workflow scheduling aims to allocate each task in the workflow to a certain time slice of computing resources for execution, in order to meet some performance criterion. As a well-known NP-hard problem, it has been a hot research topic in the distributed computing community for many years [2]. Although a great number of algorithms have been proposed to minimize the total time of executing workflows, these algorithms cannot be directly applied to the cloud environment [3], [4]: in clouds, users should first select appropriate types and amount of services from the 'infinite' cloud resources to execute workflows; the cloud provider offers heterogeneous resources with various processing capabilities and costs, and the total usage cost of cloud services is also a critical user requirement besides execution time.

Faster cloud services are usually more expensive than slower ones, and therefore users face a time-cost tradeoff in selecting services. A general way to address this trade-off is to minimize monetary cost under a deadline constraint. Up to present, only a few approaches have been presented to address this problem in the literature [5], [6], [7], [8], [9], [10] and they can be divided into two categories: local heuristics [5], [6] and metaheuristics [7], [8], [9], [10]. The former employs a simple heuristic based on a deadline distribution and can fall into a locally optimal solution easily. By contrast, existing metaheuristic methods do not exploit the characteristics of the scheduling problem fully to conduct an effective search. Furthermore, existing methods pay little attention to the situation where the sub-deadline of a task is not met during the scheduling procedure, which is not equivalent to violation of the workflow deadline.

This paper proposes a simple heuristic ProLiS and a metaheuristic algorithm L-ACO for deadline-constrained workflow scheduling. ProLiS distributes the deadline to each task based on a novel definition of probabilistic upward rank and then follows a two-step list scheduling method: order tasks according to probabilistic upward rank and then sequentially

- *Q. Wu, Q. Zhu, and Y. Xia are with the College of Computer Science, Chongqing University, Chongqing 400044, China. E-mail: {wqw, qszhu}@cqu.edu.cn, xiayunni@hotmail.com.*
- *F. Ishikawa is with the National Institute of Informatics, Tokyo 1010003, Japan. E-mail: f-ishikawa@nii.ac.jp.*
- *J. Wen is with the College of Software Engineering, Chongqing University, Chongqing 400044, China. E-mail: jhwen@cqu.edu.cn.*

Manuscript received 5 Oct. 2016; revised 26 June 2017; accepted 30 July 2017. Date of publication 3 Aug. 2017; date of current version 15 Nov. 2017. (Corresponding author: Quanwang Wu.)
Recommended for acceptance by R. Prodan.
For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.
Digital Object Identifier no. 10.1109/TPDS.2017.2735400

1045-9219 © 2017 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

allocate each task to a service that meets the sub-deadline and minimizes the cost. L-ACO carries out the deadline-constrained cost optimization based on ant colony optimization and ProLiS: the ants construct an ordered task list according to a pheromone trail and the probabilistic upward rank of a task, and then use the same deadline distribution and service selection methods as ProLiS to build solutions. The MMAS (Min Max Ant System) framework is utilized for the pheromone updating in L-ACO. Moreover, in order to guide the search towards a near-optimal solution meeting the deadline, the deadline constraint is relaxed and this relaxation is gradually diminished until is removed completely. Correspondingly, an ε comparison between solutions is introduced. In the experiments, the proposed approaches are compared with the two most-cited ones from the literature (i.e., IC-PCP and PSO), and the experimental results reveal that the performance of ProLiS is very competitive and in each setting L-ACO attains the highest success ratio of meeting deadlines, and the least execution cost.

The remainder of this paper is organized as follows: Section 2 reviews the related work. Section 3 formulates the deadline-constrained cost optimization problem for cloud workflow applications, while Section 4 presents the optimization approaches. Empirical studies are shown in Section 5. Finally, Section 6 concludes our work and points out future work.

2 Related Work

Workflow scheduling problems have been investigated extensively over the past several decades. How to best schedule workflows on a multiprocessor and in a grid system are surveyed in [11] and [12], respectively. In a traditional distributed system, workflow scheduling only focuses on how to minimize the total execution time of a workflow and there are basically three kinds of scheduling algorithms to achieve this objective: list scheduling, cluster scheduling, and duplication scheduling. Among them, list scheduling is the most widely-used heuristic (e.g., HEFT [13] and PEFT [14]) and it consists of two steps: construct a scheduling list (i.e., a sequence of tasks) by assigning tasks priorities and then sequentially allocate each task from this list to the resource which allows for the earliest starting time.

In the cloud environment, the usage cost is also a crucial user requirement and thus constrained workflow scheduling is required by realistic applications. Deadline-constrained and budget-constrained workflow scheduling are the two main categories of this literature. We review these in turn and then discuss workflow scheduling with other objectives.

2.1 Deadline-Constrained Workflow Scheduling

To tackle deadline constrained workflow scheduling, *deadline distribution* is the most widely-used method. Yu et al. [6] partition tasks into different levels on the basis of their synchronization properties and divide the overall deadline to task partitions in proportion to their minimum execution time. Resources from the utility computing environment are selected for partitions to meet the sub-dealine. A drawback of this work is that it does not distinguish communication time of a task from its execution time. Abrishami et al. introduce the concept of Partial Critical Path (PCP) in [15] and propose two algorithms called IC-PCP and IC-PCPD2 for scheduling workflows in the cloud based on PCP [5]. The former distributes the overall deadline to PCPs, while the latter further distributes the deadline to each task in proportion to its minimum execution time. Then, the cheapest resource which can meet the latest finishing time of the task is selected for partitions and tasks, respectively.

To mitigate effects of performance variation of resources, Calheiros et al. [16] propose the EIPR algorithm (Enhanced IC-PCP with Replication) to use idle time of provisioned resources and budget surplus for task replication. In [17] and [18], the authors focus on deadline-constrained workflow scheduling as well, but they have additional assumptions in the scheduling models: [17] assumes that tasks in the workflows are grouped into levels of identical tasks and [18] consider homogeneous resources which can work on different CPU frequencies.

Besides the deadline distribution based heuristic methods, metaheuristics are also utilized to address this problem. For example, Rodriguez and Buyya [7] solve this constrained optimization problem by using particle swarm optimization: a particle's position is encoded to represent the mapping between tasks and resources, and then a schedule generation algorithm is used to convert a particle's position into a schedule. Other metaheuristics such as ant colony optimization (ACO), genetic algorithms and symbiotic organism search are used in [8], [9] and [10], respectively. However, these methods only capture mappings between tasks and resources in the encoding of the metaheuristics and ignore the task order, which can be critical for the scheduling problem.

Although there are some other studies in the literature focusing on how to leverage ACO for workflow scheduling, these models are not the same as that in this paper. For example, Chen et al. in [19] propose an ACO algorithm to schedule large-scale workflows in the grid computing environment with various QoS (Quality of Service) parameters. In their work, a resource is limited in that it can only execute one task instead of multiple tasks. In [20], Lo et al. propose an enhanced ant colony system for multiprocessor system scheduling but communication time between tasks is not taken into account. Tawfeek et al. use ACO for cloud task scheduling, but the precedence dependency between tasks is not considered in their scheduling model [21].

2.2 Budget-Constrained Workflow Scheduling

Budget-constrained scheduling aims to finish a workflow as fast as possible within a given budget. BHEFT [22] extends the HEFT algorithm to tackle this constrained optimization problem: it distributes the overall user-defined budget to each task in proportion to its average execution time before scheduling and then selects the resource that allows for the earliest finish time with cost not larger than the assigned sub-budget. HBCS [23] works just like BHEFT, except that an attribute called worthiness is defined based on the normalized execution time and cost and is used for resource selection. Researchers in [24] rigorously prove that the budget-constrained scheduling problem is not only NP-complete but also non-approximable, and design a heuristic solution for this problem. Wu et al. [25] propose a heuristic algorithm called PCP-B^2 and the budget distribution is

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

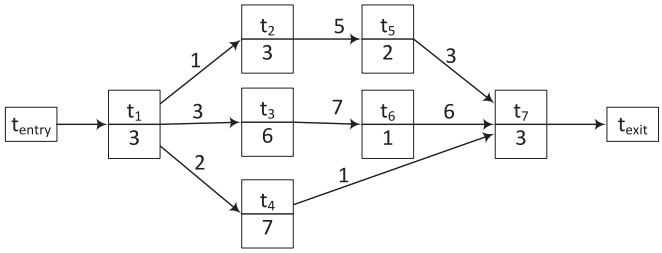

Fig. 1. An Example of Workflow Applications.

implemented via a binary search method. The advantage of the above algorithms is low time complexity.

Another idea for this problem is to first construct a schedule that has good performance on one considered objective, and then keep swapping tasks between resources to improve as much as possible for the other objective. For example, the LOSS approach [26] starts with a schedule with relatively short execution time, and repeats reassigning tasks to cheaper resources until the overall cost is lower than the given budget.

2.3 Workflow Scheduling with Other Objectives

Besides deadline-constrained and budget-constrained workflow scheduling, some researchers investigate how to achieve multi-objective workflow scheduling in clouds. For example, in [27], Zhu et al. model the workflow scheduling problem which optimizes both makespan and cost and propose an evolutionary multi-objective optimization algorithm to solve this scheduling problem. Durillo et al. propose a Pareto-based list scheduling heuristic called MOHEFT which provides the user with a set of Pareto optimal solutions [28], [29]. The MOHEFT algorithm is also extended to federated commercial clouds in [30].

In [31], Malawski et al. consider scientific workflow ensembles in IaaS clouds, and their goal is to complete as many high-priority workflows as possible given a fixed budget and deadline.

3 Scheduling Model and Problem Formulation

In this section, we first discuss our model of cloud-based workflow scheduling and then formulate the problem we consider.

3.1 Scheduling Model

A workflow application can be represented by a directed acyclic graph, DAG = (V, E), where, V is a set of n tasks $\{t_1, t_2, \ldots, t_n\}$, and E is a set of precedence dependencies. Each task t_i represents an indivisible individual application with a certain amount of computation workload w_i. A precedence dependency $e_{i,j} = (t_i, t_j)$ indicates that task t_j can start executing only after task t_i finishes. Furthermore, if there is data transmission $data_{i,j}$ attached onto $e_{i,j}$, then t_j can start only after the data from t_i has been received. The source and the destination of a dependency $e_{i,j}$ are called the *parent* task and the *child* task, respectively. To generalize the workflow with one entry and one exit, two dummy tasks t_{entry} and t_{exit} with zero execution time are added to the beginning and the end of the workflow, respectively. A sample workflow is shown in Fig. 1.

In the cloud computing model, an IaaS provider offers to its clients 'infinite' virtualized computation services $S = \{s_l\}$ with different processing capabilities, and different costs. Cloud services in this paper are used interchangeably with cloud resources to refer to the entity where the task is executed. Services with higher computational capabilities are assumed to be of higher costs. Generally, there are three pricing models for cloud services: on-demand, reserved and spot, such as in Amazon EC2 [32]. The on-demand and reserved models use fixed prices while the spot model uses dynamic prices based on bidding. In this paper, we only consider the pay-as-you-go on-demand model, as it is the most common one. According to the on-demand pricing model, users are charged on the basis of the number of time intervals that they have used the resource and any partial utilization is rounded to a full time interval (e.g., 0.3 hour is rounded to 1 hour). For service s_l, we use $c(s_l)$ to represent its cost of each time interval TI, and $p(s_l)$ to represent its processing capability.

When task t_i is allocated to service s_l, the execution time can be calculated via,

$$ET_{i,l} = w_i/p(s_l). \tag{1}$$

Moreover, if s_l is leased by the user from LST_l (lease start time) to LFT_l (lease finish time), the required cost can be calculated via,

$$EC_l = \lceil (LFT_l - LST_l)/TI \rceil \times c(s_l). \tag{2}$$

We assume that all services of the provider are in the same physical region, so the average bandwidth (bw) between the computation services is roughly equal and the internal data transfer is free. Thus, as shown in (3), the data transfer time of a dependency $e_{i,j}$, $TT_{i,j}$, depends on the amount of data to be transferred and when both tasks t_i and t_j are executed on the same service, $TT_{i,j}$ becomes zero,

$$TT_{i,j} = \begin{cases} data_{i,j}/bw & if\ srv(t_i) \neq srv(t_j) \\ 0 & otherwise \end{cases}. \tag{3}$$

Let RT_l be the earliest time at which service s_l is ready to execute t_i. The start time $ST_{i,l}$ and finish time $FT_{i,l}$ of task t_i on s_l can be calculated by,

$$ST_{i,l} = \max\left\{RT_l, \max_{t_j \in t_i's\ parents} \{FT_j + TT_{j,i}\}\right\} \tag{4}$$

$$FT_{i,l} = ST_{i,l} + ET_{i,l}. \tag{5}$$

3.2 Problem Formulation

In general, workflow scheduling aims to both assign tasks to resources and to order their execution. Based on the model defined above, a schedule can be denoted as $<R, M>$. $R = \{s_1, s_2, \ldots, s_{|R|}\}$ is a set of used resources and each s_l is associated with a service type as well as LST_l and LFT_l. M consists of all task-resource mappings, and each mapping $m = <t_i, s_l, ST_{i,l}, FT_{i,l}>$ represents task t_i is allocated to resource s_l, during the period from $ST_{i,l}$ to $FT_{i,l}$. The

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

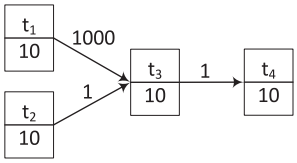

Fig. 2. A Workflow Example for Illustration.

execution time $\varphi(\mathrm{R}, \mathrm{M})$ of a workflow application (a.k.a., *makespan*) is the overall schedule length and can be calculated via,

$$\varphi(R, M) = \max_{i \in V}\{FT_i\}. \tag{6}$$

Meanwhile, its total cost $f(R, M)$ can be obtained via,

$$f(R, M) = \sum\nolimits_l EC_l. \tag{7}$$

Let D be the user-specified deadline. Then, the deadline-constrained cost optimization problem for workflow applications in clouds is defined as:

$$\begin{aligned} &min. \quad f(R, M) \\ &s.t. \quad \varphi(R, M) \leq D \end{aligned} \tag{8}$$

On the basis of the widely-accepted Pinedo's three-element notation framework for scheduling problems [2], the problem described above can be denoted as '$Q\infty$ | *prec*,s_{ij} | *makespan, cost*', where '$Q\infty$' represents infinite resources in parallel with different speeds and costs, '*prec*, s_{ij}' represents tasks associated with precedence constraints and sequence dependent setup time, and '*makespan, cost*' indicates that the considered objectives include execution time and cost.

4 Optimization Approaches

In this section, we propose a simple probabilistic list scheduling method ProLiS and a metaheuristic algorithm L-ACO for deadline-constrained cost optimization.

4.1 Deadline-constrained Probabilistic List Scheduling (ProLiS)

Traditional list scheduling methods for makespan minimizing consist primarily of two steps: construct an ordered task list, and sequentially allocate each task from the list to services. Since we focus on deadline-constrained cost optimization, the proposed ProLiS includes an extra step to first distribute the whole deadline D to each task of the workflow. Then, in the service selection step, it assigns each task to the service which meets its sub-deadline and minimizes the cost.

4.1.1 Deadline Distribution Based on Probabilistic Upward Rank

Let upward rank of a task r_i be its longest path from it to t_{exit} and it can be recursively defined as,

$$r_i = \max_{t_j \in t_i's\ children}\{r_j + data_{i,j}/bw\} + w_i/p(s^*). \tag{9}$$

We choose the fastest service s^* here because the 'infinite' IaaS services in clouds are heterogeneous, and the first objective of the considered scheduling problem is to satisfy the deadline. The upward rank can be calculated by traversing the graph upward from t_{exit}, and the upward rank of t_{entry} (denoted as r_{entry}) is the length of the workflow's critical path.

An intuitive deadline distribution method is to set a sub-deadline of a task proportionally to its longest path from the entry node to it, i.e., $r_{entry} - r_i + w_i/p(s^*)$, and calculate the sub-deadline of task t_i via,

$$sd_i = D \times \frac{r_{entry} - r_i + w_i/p(s^*)}{r_{entry}}. \tag{10}$$

This method is in essence similar to traditional deadline distribution methods for workflows, relying on ranking properties for scheduling, although they differ in concrete implementation. Examples of such differences include task partitions in [6], partial critical path in [5], [15] and minimum dependencies in [33]. However, distribution methods of this kind do not take into account the fact that when both tasks are deployed on the same service, data transmission time between them becomes zero. This may result in an unfair deadline distribution. For example, suppose the deadline for the workflow in Fig. 2 is 103.1, distribution methods of this kind will set the sub-deadline of t_1, t_2, t_3, t_4 to 1, 1, 102 and 103.1, respectively. The deadline proportion t_3 gains is much larger than the other tasks even though the execution time of each task is the same. This may result in the other three tasks not meeting their sub-deadlines, even when assigned to the fastest service.

If, we consider that during the service selection stage it is quite possible that t_1 and t_3 will be assigned to the same service because of the much larger transmission time between them, the resulting deadline distribution will be more reasonable. Therefore, we introduce as a heuristic for the deadline distribution the probabilistic upward rank of a task pr_i as follows:

$$pr_i = \max_{t_j \in t_i's\ children}\{pr_j + \gamma_j \times data_{i,j}/bw\} + w_i/p(s^*). \tag{11}$$

Here, γ_j is a Boolean variable denoting whether transmission time to t_j is considered in the calculation of pr_i. Specifically, we have:

$$\gamma_j = \begin{cases} 0, & if\ 1 - \theta^{-ccr_j} < rand() \\ 1 & otherwise \end{cases} \tag{12}$$

Here, ccr_j is the computation to communication ratio of t_j, i.e., $(w_j/p(s^*))\,/\,(data_{i,j}/bw)$, $rand()$ is a function returning a random number in [0, 1), and θ is a parameter larger than 1. Therefore, the less ccr_j is, the larger the probability that γ_j returns 0 is, and vice versa.

A larger $\boldsymbol{\theta}$ indicates the tendency to focus on the transmission time is stronger. If $\boldsymbol{\theta}$ is infinitely large, the transmission time is always considered and the proposed concept will be basically the same as upward rank (or b-level in some literature). If it is set to 1, transmission time is never considered and the concept of the probabilistic upward rank will basically be the same as the static level [11].

Deadline distribution based on probabilistic upward rank is implemented via (13), where pr_i is used instead of r_i,

$$psd_i = D \times \frac{pr_{entry} - pr_i + w_i/p(s^*)}{pr_{entry}}. \tag{13}$$

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

4.1.2 Task Ordering and Service Selection

Although there are a number of properties to rank tasks, such as upward rank, static level, and ALAP (As-Late-As-Possible) [11], we choose to use the proposed probabilistic upward rank to order tasks here as it is aware of the fact that data transmission time may become zero.

The detailed service selection algorithm is shown in Algorithm 1. The service candidates include all the services that have been used in the solution (i.e., R), and those that have not been used but can be added to R at any time (one service for each type). As we mentioned above, the first criterion for service selection is to select a service which meets its sub-deadline and minimizes the cost increment of adding t_i (Line 3). Note that this increment is not directly calculated as the cost of running t_i on s_l, but is calculated as the execution cost of s_l after adding t_i minus that before adding t_i.

However, in the case when no service can meet the sub-deadline, the criterion to select a service from R is instead to minimize the finish time of the task (Line 5). Moreover, if the selected service is not of the fastest type, we try to set its type to a faster level and update the finish time of each task deployed on it (Lines 6-9). By doing so, the probability for the solution to meet the overall deadline is increased.

Algorithm 1. Service Selection in ProLiS

1: $R \leftarrow \emptyset, \mathrm{M} \leftarrow \emptyset$
2: **for** each task t_i in the task ordering **do**
3: $s_l \leftarrow$ select a service that meets psd_i and minimizes the cost increment of adding t_i
4: **if** s_l is *null* **then**
5: $s_l \leftarrow$ select a service which allows earliest finish time
6: **while** psd_i is not met && s_l is not of the fastest type **do**
7: upgrade the type of s_l to a faster level
8: update finish time of tasks on s_l
9: **end while**
10: **end if**
11: calculate $ST_{i,l}$, $FT_{i,l}$ via (4) and (5), respectively
12: $\mathrm{M} \leftarrow \mathrm{M} \cup \{ < t_i,\ s_l,\ ST_{i,l},\ FT_{i,l} > \}$
13: **if** $s_l \notin R$ **then**
14: $R \leftarrow R \cup \{s_l\}$
15: **end if**
16: **end for**
17: **return** $< R, M >$

In addition, the leasing period of each service s_l in R starts when the first task t_u deployed on s_l begins to receive data from its parents, and ends when the last task t_v deployed on s_l finishes transferring data to its children, as shown in,

$$LST_l = ST_{u,l} - \max_{t_k \in t_u's\ parents} \{TT_{k,u}\} \tag{14}$$

$$LET_l = FT_{v,l} + \max_{t_k \in t_v's\ children} \{TT_{v,k}\}. \tag{15}$$

In the schedule obtained via Algorithm 1, a service can only be leased and released for one time from LST_j to LET_j. In reality, it is possible for one IaaS service to be leased and released several times in one schedule. In this case, the service will be regarded as multiple services here (one for each lease period) but the nature of the computations of cost and time is not changed. Further, note that because the core properties pr_i ProLiS depends on are probabilistic, the schedule produced by the method may be different each time it is used.

4.1.3 Computational Complexity

In ProLiS, the deadline distribution step needs to traverse each task and precedence dependency in the workflow graph to calculate pr_i. Given a workflow with n tasks, the maximum number of dependencies is $n(n-1)/2$, and therefore, the complexity of the deadline distribution step is $\mathrm{O}(n^2)$. The task ordering step requires a complexity of $\mathrm{O}(n^* \log n)$. In the service selection step, the service candidates are traversed during the loop of tasks (Line 3 in Algorithm 1). The upper bound of the number of service candidates is $n + |T|$, where T is the set of service types, and hence the complexity Lines 2-3 contribute is $\mathrm{O}(n^*(n + |T|))$. Because in one schedule, one service can be at most upgraded $|T|$ times, the complexity Lines 6-9 in Algorithm 1 contributes is $\mathrm{O}(n^*|T|)$. Hence, the complexity of the service selection step is $\mathrm{O}(n^*(n + |T|))$. After combining the three steps, the computational complexity of ProLiS is also $\mathrm{O}(n^*(n + |T|))$.

4.2 L-ACO

Ant Colony Optimization (ACO) is inspired by the capability of real ants to find the shortest path between their nest and a food source. It is one of the most popular metaheuristic algorithms [34]. This section describes the proposed L-ACO algorithm, which employs the ACO metaheuristic to refine the task ordering step in ProLiS to solve the workflow scheduling problem.

The full L-ACO algorithm is provided in Algorithm 2. In the beginning, an ant colony with a size of *colSize* is created and the pheromone trail is initialized. Afterwards, each ant builds a solution for the problem, the local best solution is stored to *lbSol* by comparing all the built solutions, and then the pheromone trail is updated. This procedure is repeated until the iteration number k increases to *maxNo* and finally the global best solution *gbSol* is returned.

Algorithm 2. L-ACO

1: create an ant colony with a size of *colSize*
2: initialize the pheromone trail
3: *gbSol* ← null, $k \leftarrow 0$
4: **while** $k++ <$ *maxNo* **do**
5: each ant in the colony builds a solution
6: *lbSol* ← get the best one via comparing solutions
7: update the pheromone trail
8: **if** *lbSol* is better than *gbSol* **then**
9: *gbSol* ← *lbSol*
10: **end if**
11: **end while**
12: **return** *gbSol*

In the following, the definitions of the pheromone trail and heuristic information of an ant in L-ACO are given, followed by a description of how an ant creates a schedule solution and how solutions are compared. Lastly, we describe how the pheromone trail is maintained and derive the computational complexity of the algorithm.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

4.2.1 Definition of Pheromone Trail and Heuristic Information

The pheromone trail is the core of the ACO algorithm, creating a positive feedback mechanism, and the quality of an ACO application largely depends on how the pheromone trail is defined [34]. Therefore, it is critical to choose a definition which conforms to the nature of the problem.

Since scheduling problems focus on how to allocate tasks to resources, an intuitive pheromone definition is to store the favorability of a task to a certain resource, as has been done in [7], [8], [9]. However, this definition fails to include ordering or temporal information in a schedule and also suffers from the fact that resources in the cloud are 'infinite'. As it is nearly impossible to integrate each characteristic of a schedule into the pheromone trail, in this paper we use the pheromone trail to focus on task ordering, which can then be interpreted as a schedule solution through heuristic methods like ProLiS. Therefore, we define the pheromone $\tau_{i,j}$ here as the desirability of selecting task t_j just after task t_i. $\tau_{i,j}$ is initialized to a uniform value and then continuously updated in the loop of ACO algorithm.

Another important ingredient of an ACO algorithm is the choice of good local heuristic information for ants, which will be used in combination with the pheromone trail to build solutions. The local heuristic information should be the direct attractiveness of a task and we choose to use the proposed probabilistic upward rank for this heuristic.

4.2.2 Building a Solution

Similar to ProLiS, each ant needs three steps to build a solution: distributing the deadline, ordering tasks and selecting services. The ant uses the same deadline distribution method and service selection method as ProLiS, but adopts a different way to order tasks: it relies on the pheromone trail and its own heuristic information to construct an ordered task list.

Algorithm 3. Task Ordering by Ants

```
1: L ← Empty list that will contain ordered tasks
2: RT ← Set of ready tasks
3: add the entry task to RT
4: while RT ≠ ∅ do
5:    t_j ← ant probabilistically removes a task from RT
6:    add t_j to the tail of L
7:    for each task t_k with a dependency e from t_j to t_k do
8:       remove edge e from the graph
9:       if t_k has no incoming dependency then
10:         add t_k into RT
11:      end if
12:   end for
13: end while
14: return L
```

In order to guarantee that the solution obtained through the service selection step does not violate precedence constraints between tasks, the ordered task list should be a topological ordering of the workflow graph. Specifically, a topological sort of a DAG is a linear ordering of all its vertices such that if DAG contains an edge (u, v), then u appears before v in the ordering [35]. To preserve the precedence dependencies, we make the ant adapt the Kahn's algorithm (a widely-used algorithm to generate topological sorts) to generate a task ordering based on the pheromone trail and heuristic information, as shown in Algorithm 3.

In Algorithm 3, RT is the set of ready tasks with no precedency constraints from other tasks, that is, tasks with no incoming edges. In Line 5, the ant in the colony selects the next task from RT with a probability which is positively related to the heuristic information of the ready task (pr_j), and the intensity of pheromone existing on the corresponding edge ($\tau_{i,j}$). Specifically, the probability for the ant to select t_j after visiting t_i is defined as,

$$prob_{i,j} = \frac{(\tau_{i,j})^{\alpha} \times (pr_j)^{\beta}}{\sum_{t_j \in RT} (\tau_{i,j})^{\alpha} \times (pr_j)^{\beta}}. \tag{16}$$

Here, $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ are positive numbers to control the influence of heuristic information of tasks and pheromone trails, respectively.

It is worth nothing that for a linear workflow, there is only one valid ordering. However, for most scientific workflows, the structure is very complex, as shown in Section 5.1 and there are an exponential number of different topological orderings.

4.2.3 Comparing Solutions Towards Constrained Optimization

In our problem, the deadline violation function can be defined as $max\{0, \varphi(R, M) - D\}$, that is, when the deadline is met ($\varphi(R, M) \leq D$), the value of this function is 0, otherwise, its value is $\varphi(R, M) - D$. In general, there are two directions in metaheuristic algorithms to handle such constrained optimization problems [36]: (1) combined optimization; (2) independent optimization. The former usually introduces a penalty coefficient to combine the objective function and the violation function for single-objective optimization. The challenge is that it is very difficult to select an appropriate value for the penalty coefficient, and in our problem neither objective nor violation functions can be easily normalized based on their extremums, which makes the combination even harder. The latter case optimizes the two functions separately via a lexicographic order in which the violation function precedes the objective function. Its drawback is that once a solution cannot meet the constraint (even if it has a very good objective value), it will be inferior to all other solutions which meet the constraint.

In this paper, inspired by the $\boldsymbol{\varepsilon}$-constraint handling method proposed for differential evolution [37], we use an improved independent optimization method to mitigate its side effects and guide the search towards solutions meeting the deadline and minimizing the cost: the deadline constraint D at the kth iteration of L-ACO is relaxed to $D_{\varepsilon}(k)$ as shown in (17), and $\boldsymbol{\varepsilon}$ comparison $>_{\varepsilon}$ between two solutions (f_1, φ_1) and (f_2, φ_2) in (18) is introduced to compare solutions,

$$D_{\varepsilon}(k) = \begin{cases} D + \max\{0, M_{base} - D\} \times (1 - k/k_T)^{cp} & 0 \leq k < k_T \\ D & k \geq k_T \end{cases} \tag{17}$$

$$(f_1, \varphi_1) >_{\varepsilon} (f_2, \varphi_2) \equiv \begin{cases} f_1 < f_2, \ if\ \varphi_1, \varphi_2 \leq D_{\varepsilon}(k) \\ f_1 < f_2, \ if\ \varphi_1 = \varphi_2 \\ \varphi_1 < \varphi_2, \ otherwise \end{cases}. \tag{18}$$

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

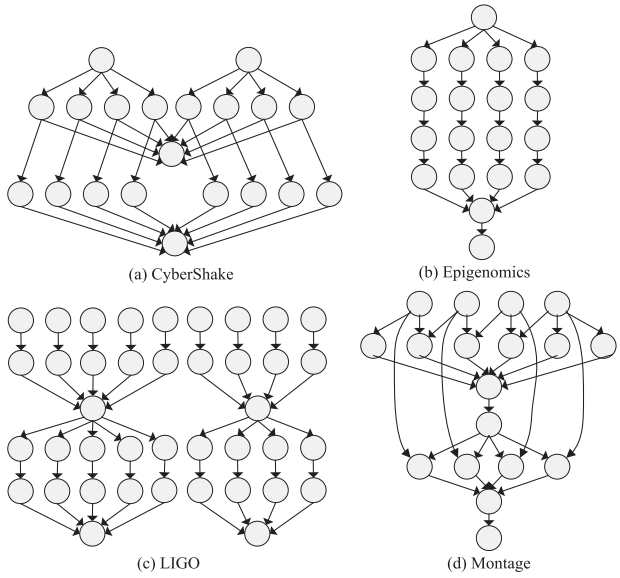

Fig. 3. The Structure of Four Realistic Scientific Workflows.

In (17), M_{base} is the makespan value when allocating all tasks to the slowest service, k is the current iteration number of L-ACO, k_T is the iteration number in which relaxation is terminated, and cp is the parameter to control the curve of $D_\varepsilon(k)$. The relaxed deadline $D_\varepsilon(k)$ decreases gradually with the increase of k until reaching k_T. Afterwards, the deadline is no longer relaxed in order to obtain solutions with no constraint violation.

By adopting $\boldsymbol{\varepsilon}$ comparison, we can see that in the early iterations, solutions violating the constraints but with small cost may be considered, and in the late iterations, solutions meeting the constraints are always preferred.

4.2.4 Updating the Pheromone Trail

To update the pheromone trail, we mainly follow the approach of Stutzle and Hoos's MAX-MIN Ant System (MMAS) [38]. We choose this version of the ACO algorithm because it is simple to understand and implement, and at the same time gives very good performance.

The pheromone values on pheromone trails are bound between an upper and lower limit (τ_{min} and τ_{max}) in order to avoid search stagnation and enhance exploration. At the beginning of L-ACO, pheromones of all edges are initialized to τ_{max}. In each iteration, the ant which builds the best solution, i.e., which ranks highest via $\boldsymbol{\varepsilon}$ comparison, deposits a certain amount of pheromone on each visited edge. Meanwhile, a percentage of existing pheromones in all edges evaporate. Thus, at the end of the kth iteration of the algorithm, the pheromone is updated according to the following formula:

$$\tau_{i,j}(k+1) = (1-\rho) \times \tau_{i,j}(k) + \Delta\tau_{i,j}(k), \tag{19}$$

where, $\boldsymbol{\rho}$ is the pheromone evaporation coefficient, and $\Delta\tau_{i,j}(k)$ is the amount of pheromone deposited by the ant that builds the best solution (denoted as $s_{best}(k)$), defined as,

$$\Delta\tau_{i,j}(k) = \begin{cases} 1/f_{best}(k) & if\ e_{i,j} \in s_{best}(k) \\ 0 & otherwise \end{cases}. \tag{20}$$

4.2.5 Complexity Analysis

We can see from Algorithm 2 that the ants build $maxNo^*colSize$ solutions in total. The task ordering procedure in Algorithm 3 the ant uses has two parts: a traversal through each task and edge in the workflow graph (Lines 4-13 except 5), and a nested loop through tasks (Lines 4-5). Thus, its upper bound on the complexity is $O(n^2)$. The service selection step in Algorithm 1 the ant uses has a time complexity of $O(n^*(n+|T|))$, and thus the computational complexity of L-ACO is $O(maxNo^*colSize^*n^*(n+|T|))$.

5 Evaluation

In this section, we first describe the experiment settings we use and then compare the proposed methods with state-of-the-art approaches. Finally, we investigate the scalability of the methods.

5.1 Experiment Settings

Since it is very difficult to perform repeatable experiments on real datacenters, we use simulation methods to evaluate the effectiveness and efficiency of our proposed approaches. This allows us to conduct a statistically significant number of experiments for a broad range of application configurations. Specifically, simulation experiments were conducted by implementing the proposed algorithms on a PC with Core i7 3.2 GHz, 8 GB RAM, Windows 10, Java 2 Standard Edition V1.7.0, and the source code is available at https://github.com/wuquanwang/workflow.

We use four diverse scientific workflow applications for performance evaluation: Cybershake, Epigenomics, LIGO, and Montage. These workflow applications differ in terms of structure, communication data, and computational characteristics. Fig. 3 depicts the structure of relatively small workflows of each application. In general, CyberShake is used to characterize earthquake hazards by generating synthetic seismograms and is a data intensive workflow with large memory and CPU requirements; Epigenomics is essentially a data processing pipeline to automate the execution of various genome sequencing operations; LIGO is used in physics for detecting gravitational waves and has CPU intensive tasks that consume large amounts of memory; Montage is an astronomy application used to generate custom mosaics of the sky and most of its tasks are I/O intensive, but do not require much CPU capacity. More details about these workflows can be found in [39]. Bharathi et al. developed a workflow generator to produce synthetic, parameterizable workflows that closely resemble these application workflows[1]. Using this workflow generator, they created workflow applications with different numbers of tasks and these workflows are available in DAX (Directed Acyclic Graph in XML) format from their website. We choose three sizes for each workflow application, which are: small (about 50 tasks), medium (about 100 tasks) and large (about 500 tasks). Moreover, for each size, 20 instances are used.

In our experiments, we assume the use of a datacenter offering 9 different types of services, each with a different processing capacity and cost as shown in Table 1. As the

1. https://confluence.pegasus.isi.edu/display/pegasus/WorkflowGenerator

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Capabilities and Costs of Available Service Types

Type	Speed	Cost	Type	Speed	Cost
$type_1$	1.0	0.12	$type_6$	3.5	0.595
$type_2$	1.5	0.195	$type_7$	4.0	0.72
$type_3$	2.0	0.28	$type_8$	4.5	0.855
$type_4$	2.5	0.375	$type_9$	5.0	1.0
$type_5$	3.0	0.48			

tasks in the above workflows are characterized by runtime on a standard computation service instead of task workloads, the speed we use to characterize services here is represented as a multiple of the speed of this standard service. The average bandwidth between services is set to 20 MBps which is the approximate average bandwidth between computation services in Amazon EC2. The time interval to charge is set to one hour, also the same as Amazon EC2.

5.1.1 Metrics

We compare ProLiS and L-ACO with IC-PCP [5] and PSO (particle swarm optimization) [7], which are the most cited works for deadline-constrained cost optimization in the literature. Because PSO, ProLiS and L-ACO are stochastic, all of them are repeated for 10 times in each case and the average performance is shown. To measure the performance of each approach for scheduling different workflows, we introduce two benchmarks:

1. Cheap Schedule. In this benchmark, only one instance of the cheapest service is used, and the workflow is scheduled by the simple and effective greedy algorithm HEFT [13]. Its makespan and required execution cost are denoted as $\mathrm{M_C}$ and $\mathrm{C_C}$, respectively;
2. Fast Schedule. In this benchmark, only the fastest type of services is used and the workflow is scheduled by HEFT as well. Its makespan and required execution cost are denoted as $\mathrm{M_F}$ and $\mathrm{C_F}$, respectively.

We introduce the deadline factor $\lambda\,(\lambda \in [0, 1])$ to represent the looseness degree of deadlines, and the deadline of a workflow is determined based on $\boldsymbol{\lambda}$ as shown in,

$$D = M_F + (M_C - M_F) \times \lambda. \tag{21}$$

TABLE 2
Parameter Setting

Parameter	Value	Description
$\boldsymbol{\alpha}$	1	Relative importance of the pheromone trail
$\boldsymbol{\beta}$	2	Relative importance of the heuristic information
$\boldsymbol{\rho}$	0.2	Pheromone evaporation coefficient
colSize	20	Number of ants in the colony
maxIter	50	Maximum number of iterations for L-ACO
k_T	35	Iteration number to terminate deadline relaxation
cp	4	Parameter to control the curve of relaxed deadline

TABLE 3
Success Ratios and Normalized Cost w.r.t. $\boldsymbol{\theta}$

$\boldsymbol{\theta}$	1	1.5	2	4	8	$+\infty$
Success ratio (%)	96.93	97.61	97.35	97.31	97.05	95.8
Normalized cost	18.32	17.65	17.85	18.65	19.14	20.65

A larger $\boldsymbol{\lambda}$ represents a looser deadline and scheduling approaches are able to select cheaper services. To evaluate the capability of each approach to obtain a valid solution meeting deadline constraints, we introduce the metric of success rate, defined as the ratio between the number of successful scheduling and the total number of scheduling. Since a large set of workflows with different attributes are used, the execution cost of a workflow is normalized by dividing by $\mathrm{C_C}$.

It is worth nothing that in the literature (e.g., [5], [7]), Fast Schedule is usually defined as scheduling each workflow task on a distinct instance of the fastest computation service, while all data transmission time is neglected. We argue that Fast Schedule defined in this way is basically an unrealistic one (unless the workflow is linear). The Fast Schedule we use is always real, that is, even when $\lambda = 0$, a schedule solution meeting the deadline exists. Hence, based on this benchmark, we can easily set a reasonably tight deadline via $\boldsymbol{\lambda}$ to test the success ratio of different approaches.

5.1.2 Parameter Setting

In [38], where the MMAS framework is proposed, the authors point out how the values of τ_{min} and τ_{max} should be set. We follow their method to dynamically set τ_{min} and τ_{max} for L-ACO and readers can refer to [38] for details. Several studies have examined how to set other ACO parameters for good performance, e.g., [40]. Parameter optimization of the ACO algorithm is problem-dependent and data-dependent and it is not the main focus of this paper. Hence, we simply enumerate a number of values for each parameter, and then, after repeated trials and comparisons, choose the combination of parameters which achieves the best performance for the experiments, as shown in Table 2.

For the parameter $\boldsymbol{\theta}$ in the definition of the probabilistic upward rank, we choose the following six values: 1, 1.5, 2, 4, 8, $+\infty$, and test the performance of ProLiS across the four different workflows with $\boldsymbol{\lambda}$ increasing from 0 to 0.2 at the step of 0.02. The average results of success ratios and normalized costs are shown in Table 3. We can see that generally ProLiS performs better when $\boldsymbol{\theta}$ is not set to the extreme values (i.e., 1 and $+\infty$). This verifies the significance of the novel concept of the probabilistic upward rank, which takes into account the transmission time probabilistically. Furthermore, ProLiS performs best when its value is 1.5, and thus we set $\boldsymbol{\theta}$ to 1.5 in the following experiments.

5.2 Results

To evaluate the ability of each approach to obtain a valid solution that meets deadline constraints, we first set $\boldsymbol{\lambda}$ to small values to have tight deadlines. Specifically, we vary $\boldsymbol{\lambda}$ from 0.005 to 0.05 with a step of 0.005. Fig. 4 depicts the average success ratio of each approach to obtain a valid solution under different configurations.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

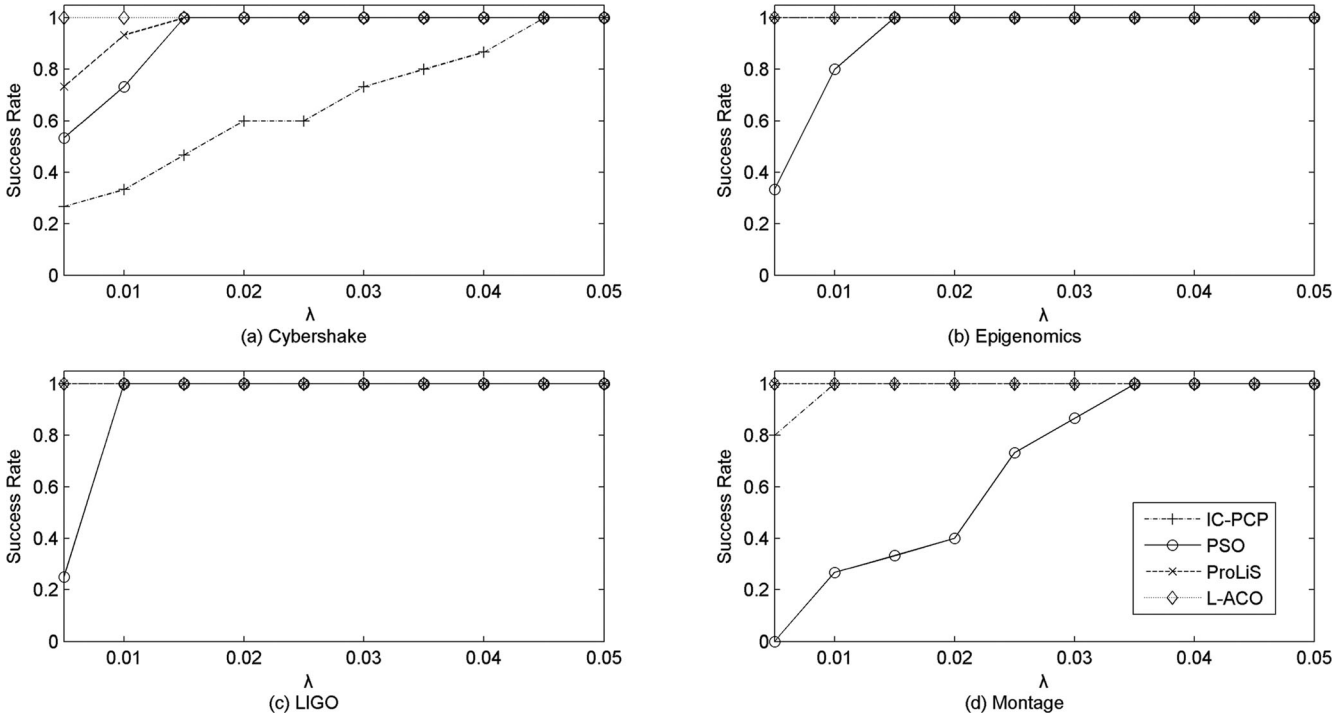

Fig. 4. The Success Ratio of Each Approach With λ Varying from 0.005 to 0.05.

We observe that when the deadline factor λ gets larger, each approach achieves a higher success ratio and when λ is 0.05, all the approaches have a success ratio of 100 percent. The performance of IC-PCP depends on the specific type of workflows: its success ratio on LIGO is 100 percent in each case while on Cybershake it is almost the worst. For each type of workflow, the success ratio of PSO is lower than 0.6 when λ is 0.005. In most cases ProLiS outperforms PSO and IC-PCP, though it may fail to obtain valid solutions when the deadline is very tight. L-ACO performs the best of all and it achieves a success ratio of 100 percent for all λ and all workflows.

In Fig. 4, when λ is very small, not all approaches can obtain valid solutions and it is meaningless to compare normalized costs when the deadline is not met. Hence, with λ varying from 0.005 to 0.05, Fig. 5 shows normalized cost of each approach only when the corresponding success ratio is

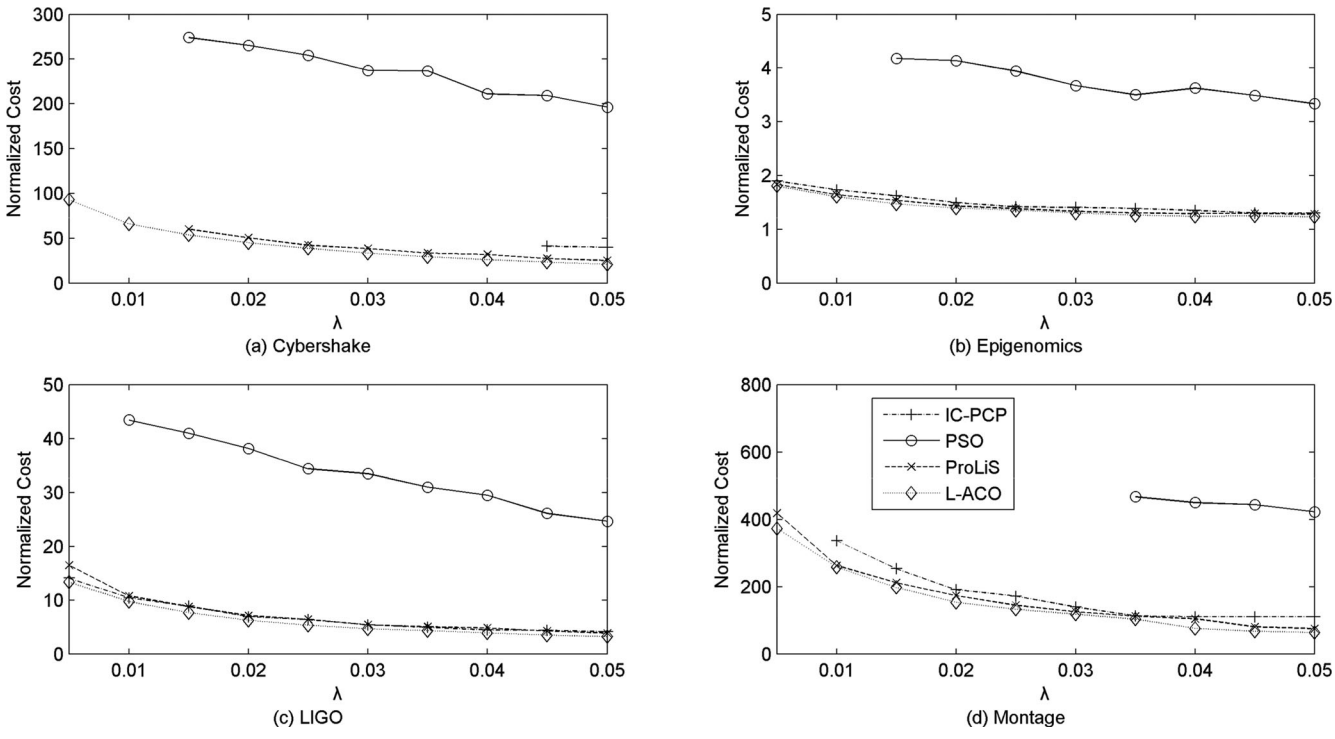

Fig. 5. The Normalized Cost of Each Approach With λ Varying from 0.005 to 0.05.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

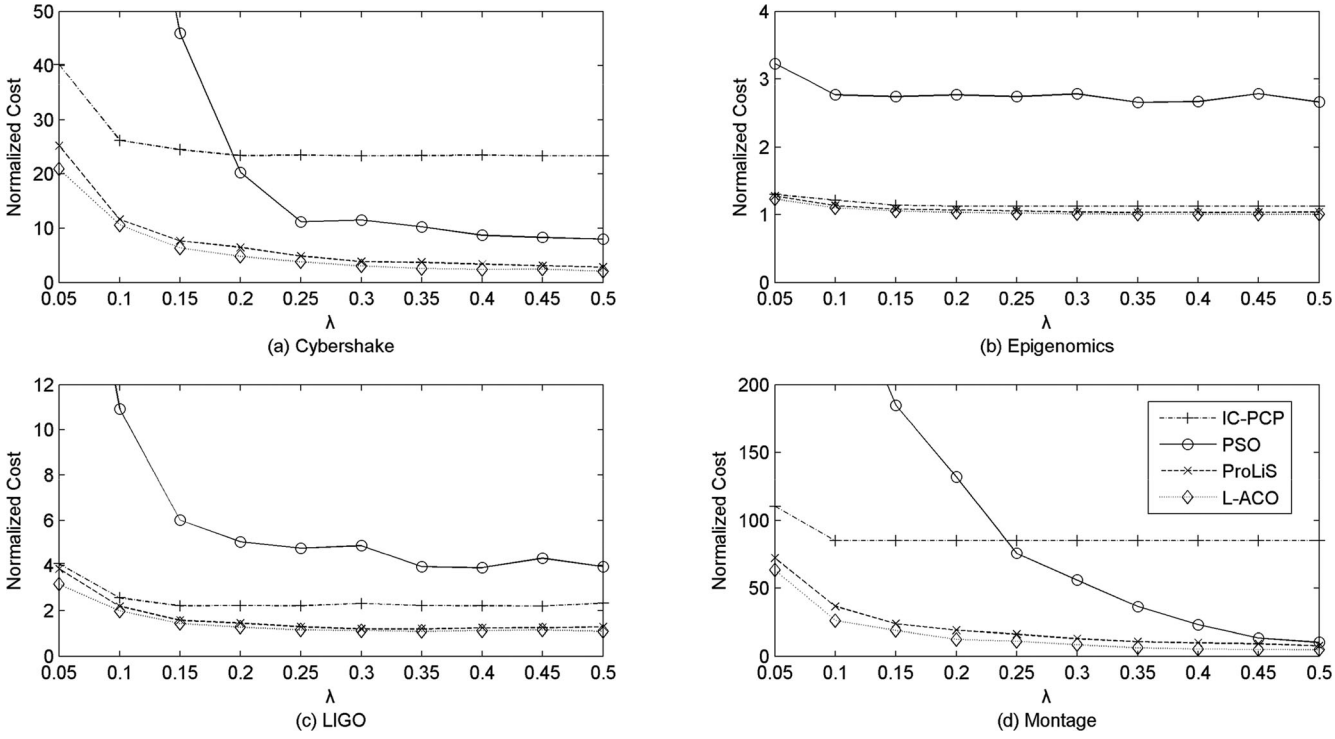

Fig. 6. The Normalized Cost of Each Approach With λ Varying from 0.05 to 0.5.

100 percent. To have a more comprehensive comparison of cost optimization performance, we also set λ to larger values and the results are depicted in Fig. 6.

The first impression we can get from Figs. 5 and 6 is that the required normalized cost decreases with rising λ. The order of magnitude of normalized cost differs largely by workflow application. Consider the Montage and CyberShake workflows in Fig. 3. There are many tasks in the second row with a short execution time. When the deadline is tight, the only way to finish all of these tasks before the single task in the third row is to assign a distinct service instance to each one. Hence, scheduling approaches have to launch many instances, while just a small fraction of their time interval is used. This is why the execution cost of the schedule for these workflows is much higher.

The performance of PSO, IC-PCP is specific to the workflow application and the value of λ. For example, IC-PCP outperforms ProLiS and PSO in Fig. 5c, but when λ is very large (e.g., larger than 0.25 in Fig. 6a), the increase of λ fails to enable IC-PCP to decrease the normalized cost, that is, the solution IC-PCP obtains is the same in spite of the increase of λ. Although the performance of PSO is promising in some cases such as Fig. 6d, it is worse than the others in most cases. The performance of ProLiS is very competitive in each case, and the more powerful approach L-ACO based on ProLiS achieves the lowest normalized cost among the four approaches. Specifically, on average, the normalized cost of L-ACO is 18.4 percent less than that of ProLiS, 37.4 percent less than that of IC-PCP and 72.6 percent less than that of PSO.

5.2.1 Discussion

IC-PCP may fail to obtain valid solutions when the deadline is very tight. This is caused by its biased deadline distribution method, as we mentioned in Section 4.1.1, and this may result in the earliest finish time of t_{exit} to be after the tight deadline. By contrast, when the deadline is very loose, IC-PCP also performs very badly. This is because it assigns a partial critical path instead of a task to a service, which prevents it from exploiting looser deadlines.

PSO does not exploit the characteristics of the scheduling problem fully to conduct an effective search. In PSO, the particles can be aware of the service type through controlling their flying directions. Nevertheless, the particles are unaware of allocating a task to a service where its parent is deployed to save transfer time or to a service whose last interval is not fully utilized to save execution cost. In addition, during schedule generation, the task ordering is fixed which hinders the search for a better solution.

The proposed ProLiS method extends list scheduling by embedding a deadline distribution and uses probabilistic upward rank to get a fairer deadline distribution. Because of its stochastic nature, it may fail to obtain a valid solution when the deadline is tight. Based on the simple and effective method ProLiS, L-ACO employs ant colony optimization to conduct a global search and achieve better performance.

5.3 Scalability

As the time complexities of ProLiS and IC-PCP are only $O(n^*(n+|T|))$ and $O(n^2)$, respectively, one might worry that the runtime of metaheuristic methods might be a bottleneck. We therefore investigate the scalability of PSO and L-ACO here. We vary the number of tasks in workflows (i.e., workflow size) from 100 to 1000 with λ set to 0.2. The average runtime of each approach is shown in Fig. 7.

As the workflow size increases, the runtime of both PSO and L-ACO increases, but L-ACO performs a bit better than PSO in terms of scalability. When the size is 1000, L-ACO

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

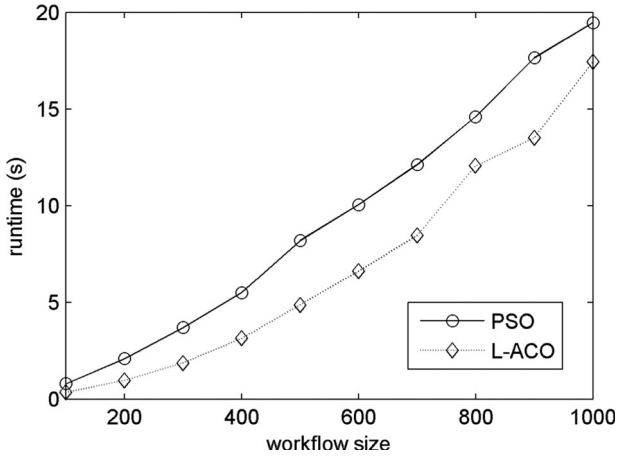

Fig. 7. Runtime of PSO and L-ACO With Regards to Workflow Size.

requires almost 17 seconds to obtain a solution. Although this overhead might be neglected when the algorithm is used offline, in real-time scheduling scenarios, L-ACO adoption decisions should take into account this runtime performance. By contrast, ProLiS will always be a good candidate even when the scheduling algorithm must find a solution quickly.

6 Conclusion and Future Work

In this paper, we have proposed two new algorithms ProLiS and L-ACO. These deploy a workflow application on cloud services in order to minimize the execution cost under a user-defined deadline constraint. In simulation experiments with real-world data, the algorithms compare favorably against state-of-the-art algorithms under both loose and tight deadline constraints. These results show that L-ACO obtains the highest success rate, and yields solutions meeting constraints that achieve the lowest costs. The results also show that the performance of the simple heuristic ProLiS is very competitive.

In future work, we are interested in how to make use of clustering and replication techniques to improve the proposed algorithms and we also plan to test them in public clouds. Besides, we intend to take into account the spot instance-based cloud services for workflow scheduling [41], which are an effective way to reduce costs when the deadline is loose. However, they introduce further challenges because of pricing dynamics and the interruption caused by bidding failures.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China under Grant 61472051 and Fundamental Research Funds for the Central Universities under Grant 0903005203386 and Grant CDJXY 0216005202069. The authors would like to thank the anonymous reviewers for their valuable comments and suggestions to improve this paper. Writing suggestions of Nan Liu from PCL are also appreciated.

References

[1] W. Van Der Aalst, and K. M. Van Hee, *Workflow Management: Models, Methods, and Systems*. Cambridge, MA, USA: MIT press, 2004.
[2] M. L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*. Berlin, Germany: Springer, 2012.
[3] F. Wu, Q. Wu, and Y. Tan, "Workflow scheduling in cloud: A survey," *J. Supercomput.*, vol. 71, no. 9, pp. 3373–3418, 2015.
[4] Z.-H. Zhan, et al., "Cloud computing resource scheduling and a survey of its evolutionary approaches," *ACM Comput. Surveys*, vol. 47, no. 4, pp. 1–33, 2015.
[5] S. Abrishami, M. Naghibzadeh, and D. H. Epema, "Deadline-constrained workflow scheduling algorithms for Infrastructure as a Service Clouds," *Future Generation Comput. Syst.*, vol. 29, no. 1, pp. 158–169, 2013.
[6] J. Yu, R. Buyya, and C. K. Tham, "Cost-based scheduling of scientific workflow applications on utility grids," in *Proc. 1st Int. Conf. e-Sci. Grid Comput.*, 2005, pp. 140–147.
[7] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioning and scheduling algorithm for scientific workflows on clouds," *IEEE Trans. Cloud Comput.*, vol. 2, no. 2, pp. 222–235, Apr. 2014.
[8] Z. G. Chen, et al., "Deadline constrained cloud computing resources scheduling through an ant colony system approach," in *Proc. Int. Conf. Cloud Comput. Res. Innovation*, 2015, pp. 112–119.
[9] Z.-G. Chen, et al., "Deadline constrained cloud computing resources scheduling for cost optimization based on dynamic objective genetic algorithm," in *Proc. IEEE Congr. Evol. Comput.*, 2015, pp. 708–714.
[10] M. Abdullahi, M. A. Ngadi, and S. I. M. Abdulhamid, "Symbiotic Organism Search optimization based task scheduling in cloud computing environment," *Future Generation Comput. Syst.*, vol. 56, pp. 640–650, 2016.
[11] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Comput. Surveys*, vol. 31, no. 4, pp. 406–471, 1999.
[12] J. Yu, R. Buyya, and K. Ramamohanarao, "Workflow scheduling algorithms for grid computing," *Metaheuristics for Scheduling in Distributed Computing Environments*. Berlin, Germany: Springer, 2008, pp. 173–214.
[13] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
[14] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014.
[15] S. Abrishami, M. Naghibzadeh, and D. H. Epema, "Cost-driven scheduling of grid workflows using partial critical paths," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 8, pp. 1400–1414, Aug. 2012.
[16] R. N. Calheiros and R. Buyya, "Meeting deadlines of scientific workflows in public clouds with tasks replication," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 7, pp. 1787–1796, Jul. 2014.
[17] M. Malawski, et al., "Scheduling multilevel deadline-constrained scientific workflows on clouds based on cost optimization," *Scientific Program.*, vol. 2015, 2015, Art. no. 5.
[18] I. Pietri and R. Sakellariou, "Cost-efficient CPU provisioning for scientific workflows on clouds," in *Proc. Int. Conf. Grid Econ. Bus. Models*, 2015, pp. 49–64.
[19] W. N. Chen and J. Zhang, "An ant colony optimization approach to a grid workflow scheduling problem with various QoS requirements," *IEEE Trans. Syst. Man Cybern. Part C Appl. Rev.*, vol. 39, no. 1, pp. 29–43, Jan. 2009.
[20] S.-T. Lo, et al., "Multiprocessor system scheduling with precedence and resource constraints using an enhanced ant colony system," *Expert Syst. Appl.*, vol. 34, no. 3, pp. 2071–2081, 2008.
[21] M. A. Tawfeek, et al., "Cloud task scheduling based on ant colony optimization," in *Proc. 8th Int. Conf. Comput. Eng. Syst.*, 2013, pp. 64–69.
[22] W. Zheng and R. Sakellariou, "Budget-deadline constrained workflow planning for admission control," *J. Grid Comput.*, vol. 11, no. 4, pp. 633–651, 2013.
[23] H. Arabnejad and J. G. Barbosa, "A budget constrained scheduling algorithm for workflow applications," *J. Grid Comput.*, vol. 12, no. 4, pp. 665–679, 2014.
[24] C. Q. Wu, et al., "End-to-end delay minimization for scientific workflows in clouds under budget constraint," *IEEE Trans. Cloud Comput.*, vol. 3, no. 2, pp. 169–181, Apr.-Jun. 2015.
[25] F. Wu, et al., "PCP-B2: Partial critical path budget balanced scheduling algorithms for scientific workflow applications," *Future Generation Comput. Syst.*, vol. 60, pp. 22–34, 2016.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.

[26] R. Sakellariou, et al., "Scheduling workflows with budget constraints," in *Proc. Integr. Res. GRID Comput.: CoreGRID Integr. Workshop*, 2007, pp. 189–202.
[27] Z. Zhu, et al., "Evolutionary multi-objective workflow scheduling in cloud," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 5, pp. 1344–1357, May 2016.
[28] J. J. Durillo, H. M. Fard, and R. Prodan, "Moheft: A multi-objective list-based method for workflow scheduling," in *Proc. IEEE 4th Int. Conf. Cloud Comput. Technol. Sci.*, 2012, pp. 185–192.
[29] J. J. Durillo, et al., "Bi-objective workflow scheduling in production clouds: Early simulation results and outlook," in *Proc. 1st Int. Workshop Sustainable Ultrascale Comput. Syst.*, 2014, pp. 19–26.
[30] J. J. Durillo, R. Prodan, and J. G. Barbosa, "Pareto tradeoff scheduling of workflows on federated commercial Clouds," *Simul. Model. Practice Theory*, vol. 58, pp. 95–111, 2015.
[31] M. Malawski, et al., "Algorithms for cost-and deadline-constrained provisioning for scientific workflow ensembles in IAAS clouds," *Future Generation Comput. Syst.*, vol. 48, pp. 1–18, 2015.
[32] Amazon Elastic Compute Cloud, 2016. [Online]. Available: http://aws.amazon.com/ec2/
[33] M. Zotkiewicz, M. Guzek, D. Kliazovich, and P. Bouvry, "Minimum dependencies energy-efficient scheduling in data centers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 12, pp. 3561–3574, Dec. 2016.
[34] M. Dorigo, *Ant Colony Optimization*. Cambridge, MA, USA: MIT Press, 2004.
[35] T. H. Cormen, et al., *Introduction to Algorithms*, 3d ed. Cambridge, MA, USA: MIT Press, pp. 1297–1305, 2009.
[36] E. Mezura-Montes and C. A. Coello Coello, "Constraint-handling in nature-inspired numerical optimization: Past, present and future," *Swarm Evol. Comput.*, vol. 1, no. 4, pp. 173–194, 2011.
[37] T. Takahama and S. Sakai, "Constrained optimization by the ε constrained differential evolution with an archive and gradient-based mutation," in *Proc. IEEE Congr. Evol. Comput.*, 2010, pp. 1–9.
[38] T. Stützle and H. H. Hoos, "MAX–MIN ant system," *Future Generation Comput. Syst.*, vol. 16, no. 8, pp. 889–914, 2000.
[39] G. Juve, et al., "Characterizing and profiling scientific workflows," *Future Generation Comput. Syst.*, vol. 29, no. 3, pp. 682–692, 2013.
[40] T. Stützle, et al., "Parameter adaptation in ant colony optimization," *Auton. Search*, pp. 191–215, 2010.
[41] P. Sharma, et al., "SpotCheck: Designing a derivative IAAS cloud on the spot market," in *Proc. 10th Eur. Conf. Comput. Syst.*, 2015, pp. 1–15.

Quanwang Wu received the BS, MS, and PhD degrees in computer science from Chongqing University in 2007, 2010, and 2013, respectively. He is currently a lecturer of Computer College in Chongqing University, Chongqing, China. He was a special researcher in the Digital Content and Media Sciences Research Division of the National Institute of Informatics (NII) in Tokyo, Japan from 2014 to 2015. His main research interests include service- oriented computing (e.g., service composition and service selection) and cloud computing (e.g., virtual machine consolidation and workflow scheduling).

Fuyuki Ishikawa received the PhD degree in information science and technology from the University of Tokyo, Japan, in 2007. He is an associate professor in the National Institute of Informatics, Japan. He also serves as an visiting associate professor with The University of Electro- Communications, Japan. His research interests include software engineering, formal methods, service-oriented computing and cyber-physical systems. He is member of the IEEE.

Qingsheng Zhu received the BS, MS, and PhD degrees in computer science from Chongqing University, Chongqing, China, in 1982, 1985, and 2000. He was a visiting scholar with the University of London, London, United Kingdom, from 1993 to 1994, and a visiting professor with the University of Illinois, Chicago, Chicago, from 2001 to 2002. Currently, he is a professor of Computer College, and the Director of the Key Laboratory of Software Theory and Technology in Chongqing University. His research interests include services computing and data mining.

Yunni Xia (SM'14) received the BS degree in computer science from Chongqing University, China, 2003, and the PhD degrees in computer science from Peking University (PKU), China, 2008. Since August 2008, he has been an associate professor in school of Computer Science at Chongqing University, China. He is the author or co-author of more than 50 research publications. His research interests are in Petri nets, software quality, performance evaluation, and cloud computing system dependability. He is a senior member of the IEEE.

Junhao Wen received the PhD degree from Chongqing University, in 2008, where he is a vice head and a professor with the School of Software Engineering. His research interests include service computing, cloud computing, and software dependable engineering. He has published more than 80 refereed journal and conference papers in the above areas. He has more than 30 research and industrial grants and developed many commercial systems and software tools.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on September 25,2023 at 13:48:16 UTC from IEEE Xplore. Restrictions apply.